

DH2323 - DGI Project

Voxel Cone Tracing

Corentin Jeanne

Corentin Jeanne <cjeanne@kth.se>

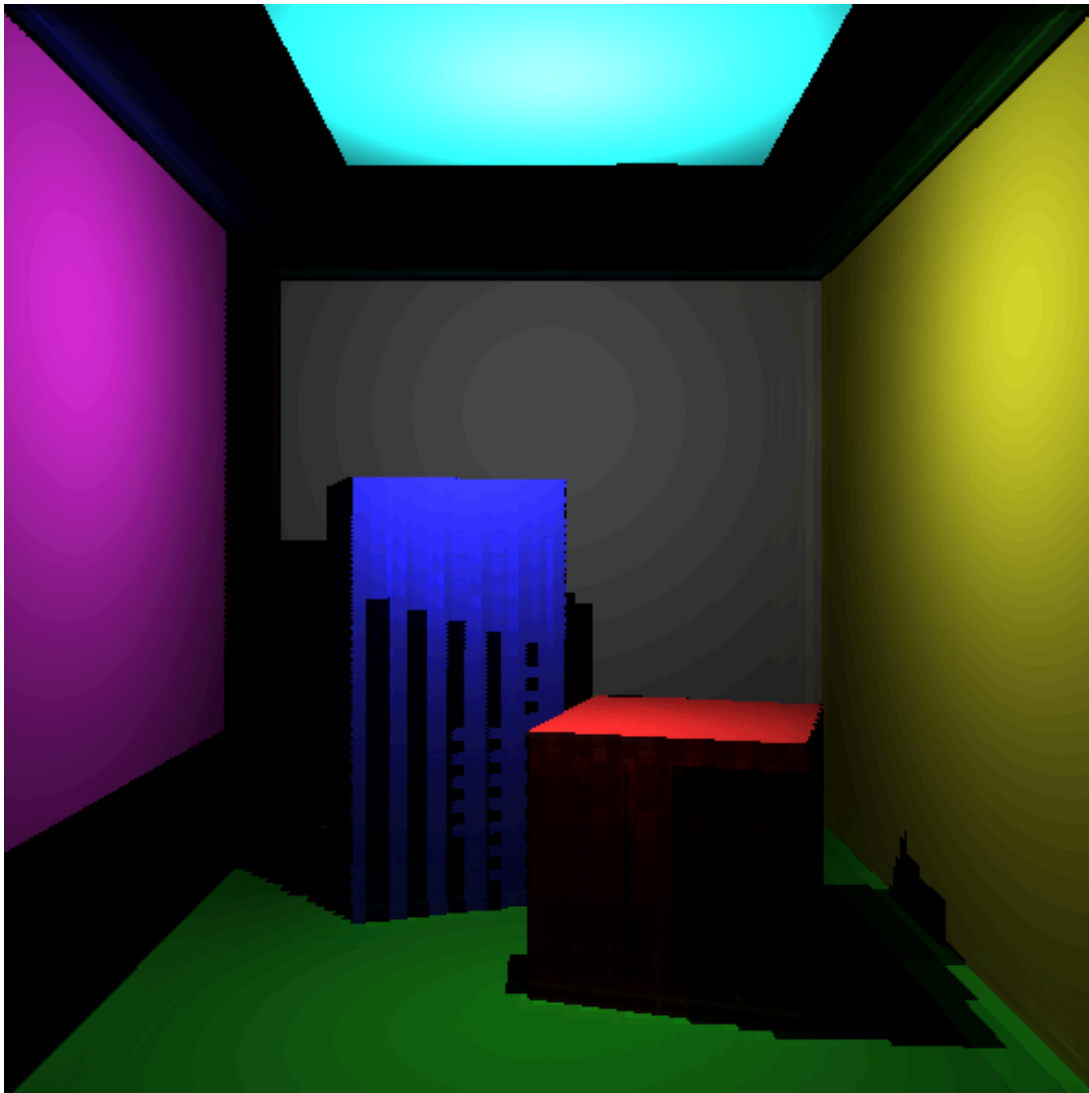


Figure 1: Rendering of the Voxel Cone Tracing algorithm implemented using SDL2

Contents

I.	Introduction	3
II.	The Voxel Cone Tracing Algorithm	3
	II.1. Computing the Shadows	3
	II.2. Voxelizing the scene	4
	II.3. Cone Tracing	4
	II.4. Final Color	5
III.	Implementation	6
	III.1. Base architecture	6
	III.2. Voxelization	7
	III.3. Shadow Mapping	7
	III.4. Cone Tracing	8
	III.5. Final color output	9
IV.	Results	9
	IV.1. Render frames	9
	IV.2. Render times	11
	IV.3. Render frames using Direct illumination only	11
	IV.4. Render Times using Direct illumination only	11
V.	Discussion	12
	V.1. Shadows	12
	V.2. Visual quality of the output	12
	V.3. Impact of the Voxel Grid resolution	12
	V.4. Render speed	12
	V.5. Memory Consumption	12
VI.	Conclusion	12
VII.	References	13

I. Introduction

Voxel Cone Tracing is a rasterization algorithm to approximate global illumination. Unlike a ray tracer, where shadows are easy to compute because light is fully simulated, a rasterizer has no direct ways to compute them as it processes one triangle at a time. Global illumination is not limited to shadows as well. It also includes ambient color from nearby objects, reflection, and glare, for example. While none of this can easily be computed by a rasterizer, it can be approximated by algorithms such as voxel cone tracing.

My goal for this project was to reimplement the algorithm in SDL2, based on the degree project *Voxel Cone Tracing Evaluation for Real-Time Applications* by Alvar Trojenborg, Filip Northman [2] and on their OpenGL implementation [5]. My final implementation is available on GitHub [6].

II. The Voxel Cone Tracing Algorithm

The Voxel Cone Tracing Algorithm (VCT) aims to approximate global illumination by first computing the correct direct light and approximating the indirect light each pixel receives.

This is done in three steps. First, a shadow map is used to generate the direct lighting data for the scene, then the scene is voxelized, and finally those voxels are used in combination with cone-tracing to compute indirect light. Each individual step is detailed below.

II.1. Computing the Shadows

The first step in VCT is to compute a shadow map. It is a destructure used to check if a point receives direct light. It consists of a depth buffer associated with a direct light. Using the depth data, we can easily check the visibility of a pixel [4].

To compute and use a shadow map, we first render the scene using a light camera and only preserve the depth buffer. The resolution of this camera can differ from the resolution of the main camera, as in Figure 2. Then, to check the visibility of a 3D point, we compute the corresponding pixel on the shadow map and realize a depth check: if the depth of the pixel is greater than the stored value, this point is behind an object from the light's point of view and is not visible; otherwise, the point is fully visible. When testing, the data structure returns a float, usually 0 or 1, where 1 means fully visible and 0 fully hidden. The value can be between when interpolating points to de-noise the texture [4].

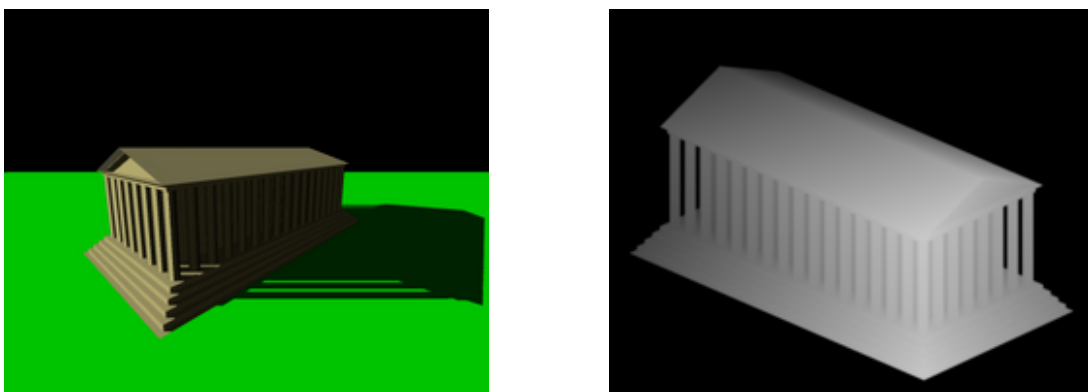


Figure 2: Full scene (left), Depth view from the light (right) [1]

Only directional light can be mapped one-to-one to a shadow map. Point lights need to split into multiple directional lights to compute a shadow cube. To check the visibility of a point, you must first get the correct map and then do the depth test.

II.2. Voxelizing the scene

The second step of VCT is to voxelize the scene and turn it into a 3-dimensional grid of voxels. Each voxel is assigned a color, taking into account the reflectance of the material and the direct light it receives. This uses the shadow map computed earlier in the process.

One easy way to voxelize the entire scene is to render it using an orthographic camera with the same resolution as the voxel grid. This way the fragment shader can be used to store the voxel information, as each pixel directly maps to a voxel. Since the fragment shader also has access to the corresponding triangle's normal and interpolated 3D position, computing the direct light of that voxel is possible.

To find out the amount of light reaching the point, we can assume a point of power p radiating light in a perfect sphere. In that case, the amount of energy e received by a point at a distance r would be $e = \frac{p}{4\pi r^2}$. The final direct light equation, taking into account the normal of the painted triangle and a point light, would be the following:

$$color = reflectance * v * \frac{\max(0, \vec{n} \cdot \vec{l}) * light}{4\pi \|pos_{light} - pos_{point}\|^2}$$

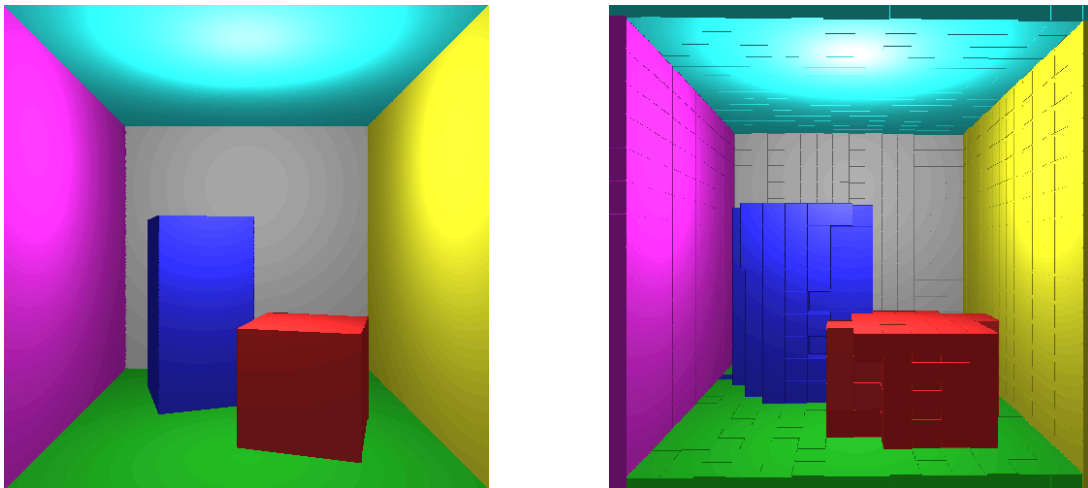


Figure 3: Example scene (left), Voxelized scene (right)

II.3. Cone Tracing

The third and final step of VCT is to use cone tracing to compute the indirect light received by a pixel. Here we take full benefit of the voxel grid by using it to quickly check for the surrounding light. This is done by casting a cone from the original position in a half-sphere around our surface, as shown in Figure 4. Each cone samples color and occlusion from the voxels on its path. The results are then combined to find the indirect illumination of a pixel.

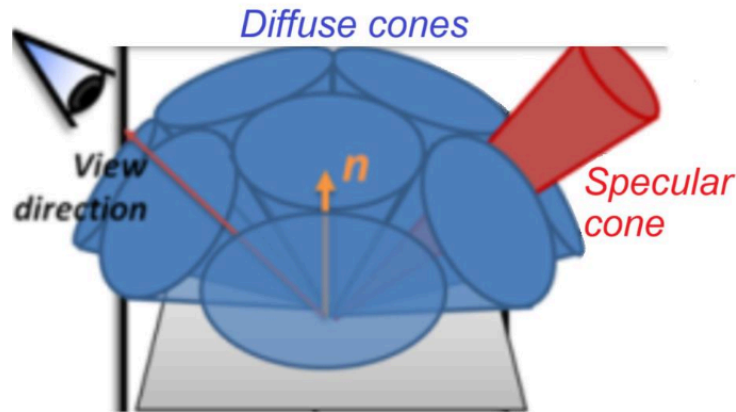


Figure 4: Multiple diffuse cones over one smaller specular cone sampling indirect diffuse and specular lighting respectively. [3]

Cones also use Level Of Detail (LOD) when sampling, as shown in Figure 5. The first simple has an LOD of 1 and samples a single voxel; the second has an LOD of 2 and samples $2 * 2 * 2 = 8$ voxels, etc. At each step, the color and occlusion are updated using front-to-back compositing using the following formulas, with c_n, a_n the color and occlusion after n samples, and c_s, a_s the sampled color and occlusion:

$$c_n = c_{i-1} + (1 - a_n) * a_s * c_s$$

$$a_n = a_{i-1} + (1 - a_n) * a_s$$

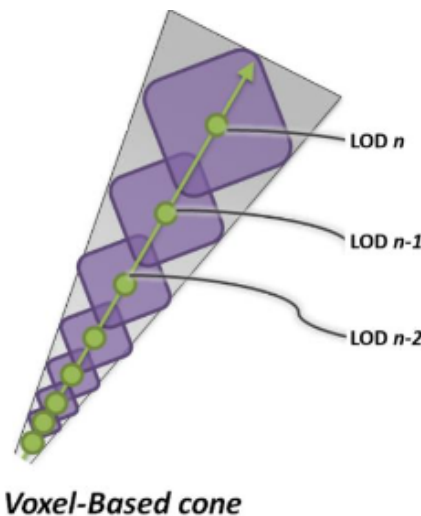


Figure 5: Cones level of details. [3]

II.4. Final Color

To get the final color of the voxel, we compose both the direct light and indirect light by the reflectance of the color.

$$color = (direct\ light + indirect\ light) * reflectance$$

The direct light can be computed using the same formula as in Section II.2, and the indirect light is found by multiplying the cone-traced color by its ambient occlusion.

III. Implementation

III.1. Base architecture

The root of the code was based on the SDL2 rasterizer implemented in lab 3. It was heavily refactored to be easier to modify. The whole rendering part is now comprised of the `Renderer` class. This class exposes an API to render points, lines, and polygons using a specific shader and render mode.

The draw mode can be either `Vertices` to only draw the points, `Wireframe` to draw the outlines, or `Default` to fill in the shapes. A `Shader` is a C++ class providing a `VertexShader` and a `FragmentShader` method. The vertex shader translates world coordinates (a `Vertex`) to screen position (a `Pixel`). The renderer then interpolates all the drawn pixels of the shapes and calls the fragment shader for each one.

The definitions of those classes are visible in Excerpt 1.

```
struct Vertex {
    glm::vec3 position;
};
struct Pixel {
    int x; int y;
    float z;
    glm::vec3 pos3d;
};

class Shader {
public:
    virtual Pixel VertexShader(const Vertex&) = 0;
    virtual void FragmentShader(const Pixel&) = 0;
};

enum class DrawMode { Default, Wireframe, Vertices, };

class Renderer {
public:
    void DrawPolygon(const std::vector<Vertex>&, Shader&, DrawMode);
    void DrawLine(Vertex, Vertex, Shader&, DrawMode);
    void DrawPoint(Vertex, Shader&, DrawMode);
};
```

Excerpt 1: Definitions from `Renderer.hpp`

When rendering, individual shaders may require additional parameters, either per triangle or globally. The global settings are set up in the function `SetupShaders` and the per-triangle settings are set before the draw call, as seen in Excerpt 2.

```
// void SetupShader():
lab3Shader.camera = &camera;
lab3Shader.light = &light;
lab3Shader.indirectLight = &ambientLight;
lab3Shader.screen = sdlAux;
lab3Shader.depthBuffer = depthBuffer;

// void Draw():
for (const auto& t : triangles) {
    lab3Shader.normal = t.normal;
    lab3Shader.reflectance = t.color;
    renderer.DrawPolygon({ {t.v0}, {t.v1}, {t.v2} }, lab3Shader, mode);
}
```

Excerpt 2: Shader setup for the Lab3 shader

Finally, the objects present in the scene, such as cameras and lights, are defined in `Objects.hpp` as shown in Excerpt 3, and the various shaders are defined in `Shaders.hpp`.

```

struct Camera {
    glm::vec3 position;
    glm::mat3 rotation;
    glm::ivec2 resolution;
    float focale;
};

struct PointLight {
    glm::vec3 position;
    glm::vec3 color;
    float intensity;
};

```

Excerpt 3: Definitions from `Objects.hpp`

III.2. Voxelization

Due to challenges when implementing the shadow map, the voxelization comes first in my implementation. The reason for this change is explained in the corresponding in Section III.3.

The voxelization is done by the class `VoxelizationShader` and follows the steps from Section II.2: rendering the scene with a custom resolution and storing the color data in a custom class, `VoxelGrid`. This class serves as easy access to the voxel data, especially as the 3D grid is stored as a 1D array.

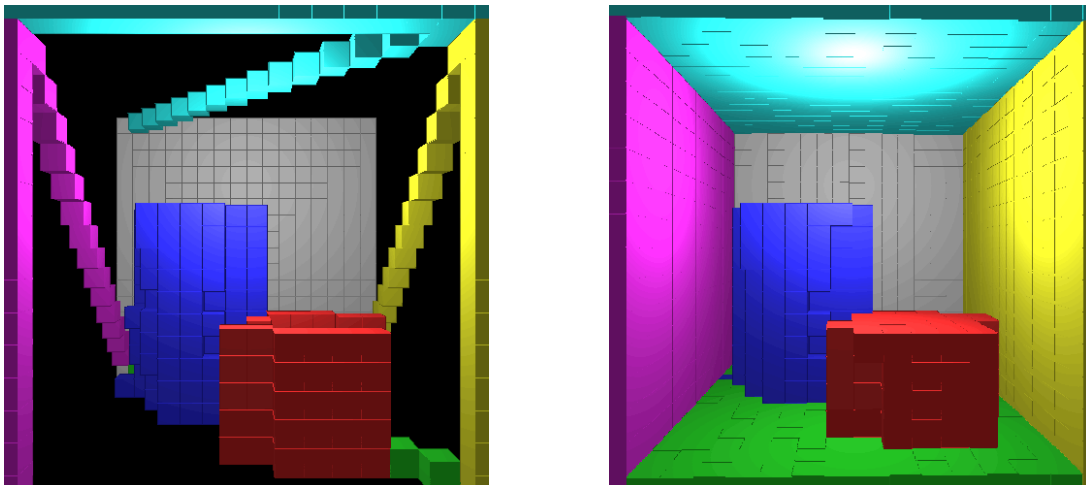


Figure 6: Voxelization without axis selection (left), voxelization using the normal's axis (right)

An important point of voxelization is choosing the correct axis when projecting the triangle, as always choosing the same axis will lead to gaps in the texture, as shown in Figure 6. An easy way to ensure good results is to project on the axis that would lead to the highest number of pixels. It can be found by using the dominant component of the normal. A normal majorly on the X-axis means that the triangle will be the most “flat” and visible when looking at it from this axis.

Since the shadows have not been computed yet, this step only stores the radiance of the voxels.

III.3. Shadow Mapping

Creating a shadow map was a challenge I did not manage to overcome in time. When I started my implementation, I prioritized having a working rendering pipeline to be able to complete the algorithm. As such, the shadows are based on raycasts using the voxel grid as obstacles and are stored per voxel. Due to this process requiring the voxels to already be computed, this step is executed after the voxelization.

Instead of storing the shadow data in a depth buffer, I opted to store it in a grid of the same dimensions as the voxel grid. This way, I could do a second voxelization pass to modify the colors. Instead of

rendering the scene from the light's point of view, I opted to render the scene in the same manner as the voxelization. This allows me to, per voxel, raycast towards the light to see if it is visible and then store this data in the shadow grid and modify the voxel color as shown in Figure 7.

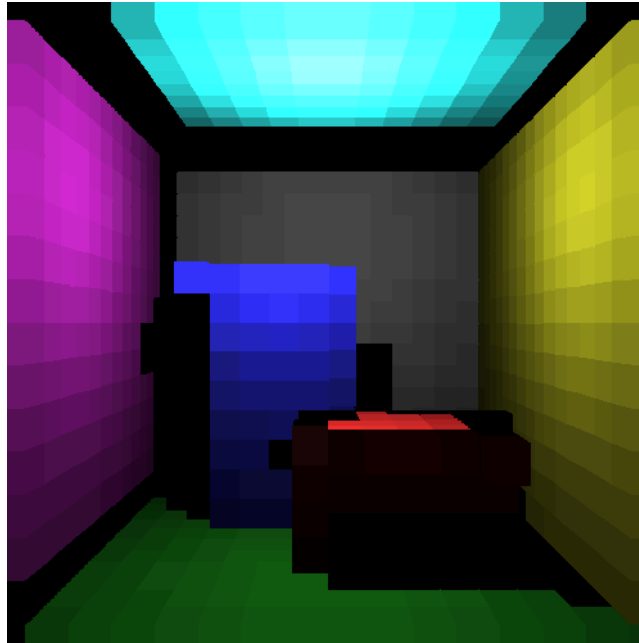


Figure 7: Voxels with shadows applied

III.4. Cone Tracing

Unlike the other two steps, Cone Tracing is not a shader, but the part of the final shader, RenderShader, responsible for computing indirect light.

The implementation of the algorithm is straight forward: iterate every cone, and for every cone step until the cone is fully occluded. To improve the quality of the results, the cone starts at an offset. This prevents the cone from hitting the source voxel and returning the wrong color. The results from the cone tracing are visible in Figure 8.

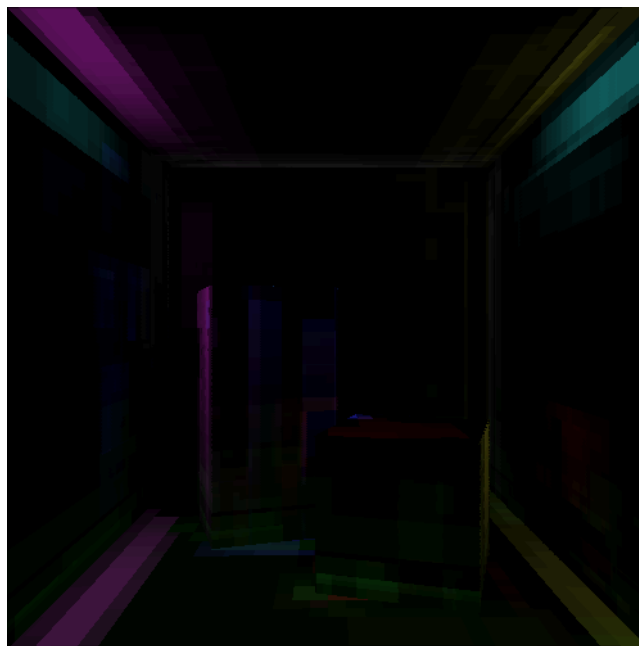


Figure 8: Indirect light resulting from cone tracing

III.5. Final color output

The final step in the process is to determine the final color of the pixel. Since both the direct light and indirect light were already computed, this task is rather simple. All that is needed is to combine both results and multiply them by the reflectance of the material. The different steps can be seen in Figure 9.

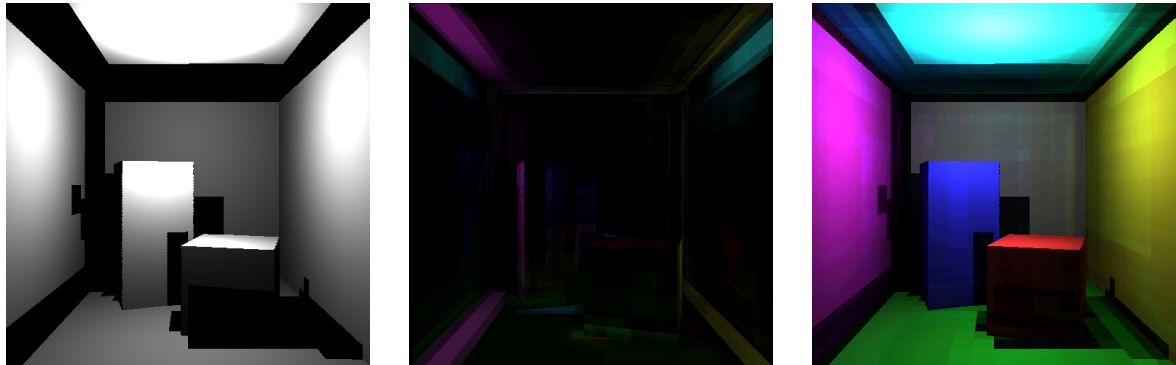


Figure 9: Direct light, indirect light, and final color output

IV. Results

IV.1. Render frames

Figure 11 compiles the render of labs 2 and 3, as well as the render of the VCT algorithm with a grid size from 8 to 64 voxels per axis and with full or half cones. When using half-cone, the same cones are cast, but with their angle divided in half to save compute time. The scene has also been remodeled in Blender, and the render is shown in Figure 10.

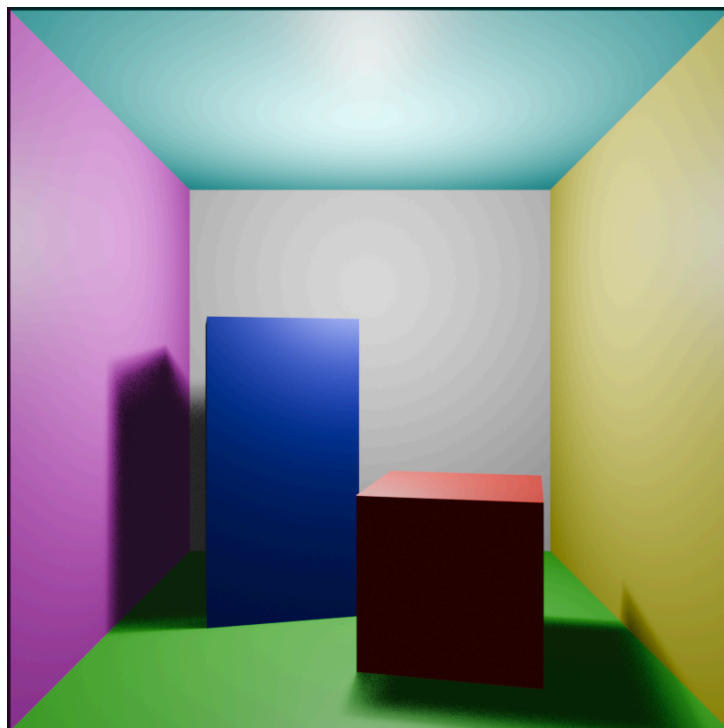
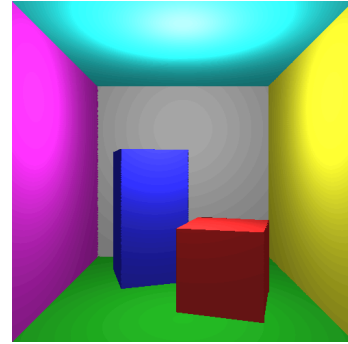
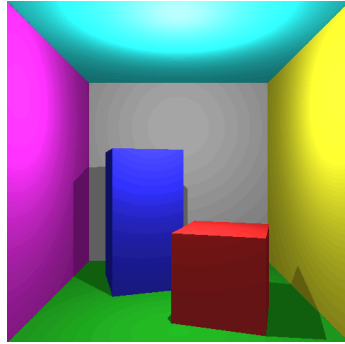
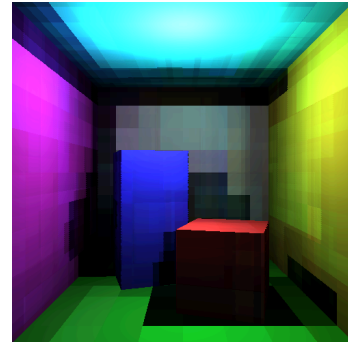
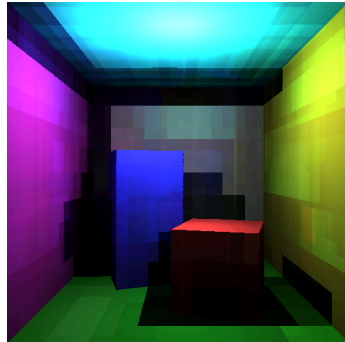


Figure 10: Render of the Scene in Blender

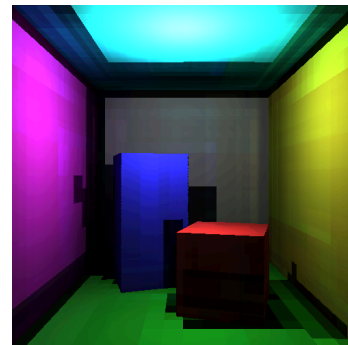
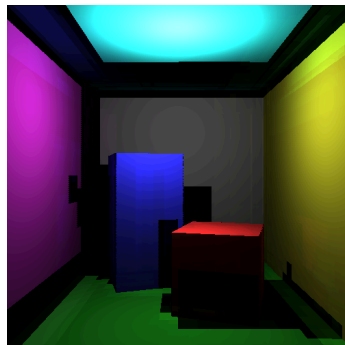
Labs 2 and 3



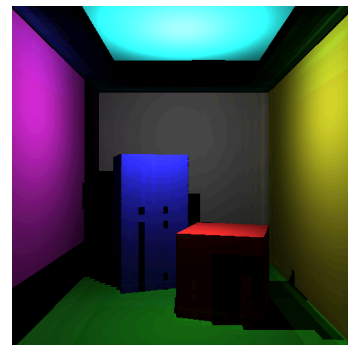
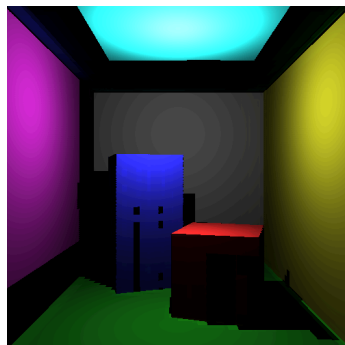
VCT 8



VCT 16



VCT 32



VCT 64

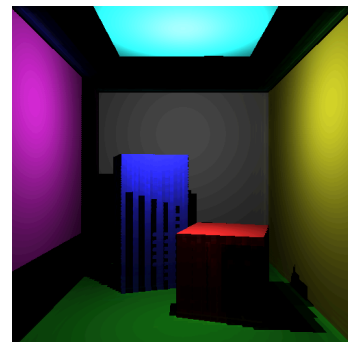
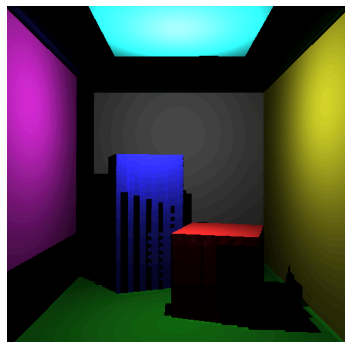


Figure 11: Render of the example scene with different configurations

IV.2. Render times

Table 1 compiles the render of labs 2 and 3, as well as the render of the VCT algorithm with 8 to 64 voxels per axis and with full or half cones. All the programs were compiled in release mode (`set(CMAKE_BUILD_TYPE Release)`) with no additional gcc flags.

Renderrer	Lab 2	Lab 3	VCT 8		VCT 16		VCT 16		VCT 16	
			Half	Full	Half	Full	Half	Full	Half	Full
Time (ms)	110	7	295	2580	374	4285	381	4839	415	5094

Table 1: Average render time of the example scene with different configurations

IV.3. Render frames using Direct illumination only

Since the labs did not include indirect illumination, I also test the renderer keeping only the direct illumination using only the lab 2 and 3 renderer, as well as VCT with grids from 8x8x8 up to 64x64x64. The results are compiled in Figure 12.

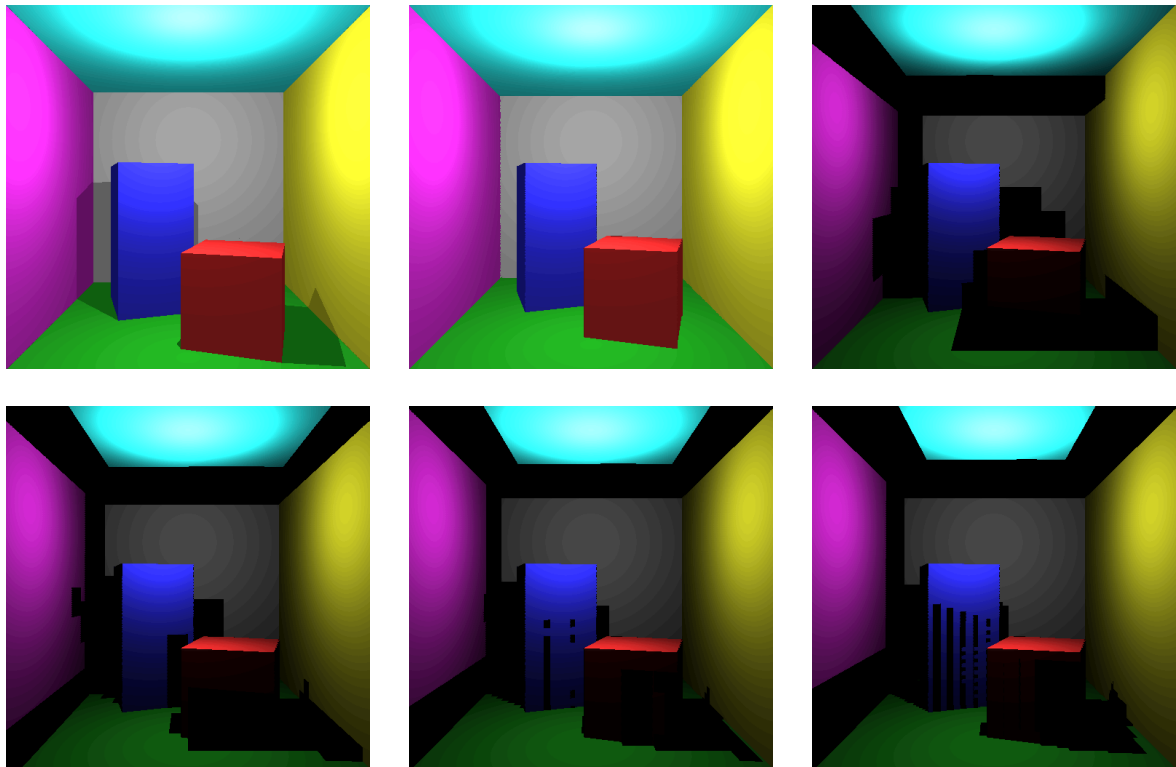


Figure 12: Render of the example scene from respectively, Lab 2, Lab 3, VCT 8, VCT 16, VCT 32, VCT 64

IV.4. Render Times using Direct illumination only

Table 2 shows the average time over 10 frames for various rendering renderers: the lab 2 and 3 renderer, as well as VCT with grids from 8x8x8 up to 64x64x64. All the programs were compiled in release mode (`set(CMAKE_BUILD_TYPE Release)`) with no additional gcc flags.

Renderrer	Lab 2	Lab 3	VCT 8	VCT 16	VCT 32	VCT 64
Time (ms)	110	7	22	26	36	57

Table 2: Average render time over 10 frames of the example scene with different configurations

V. Discussion

V.1. Shadows

The logical next step in this implementation is to finalize the implementation of a shadow map and cube. The process was already well on its way, and the code has been pushed to the branch `feat/shadow-map`. The issues I encountered were performance issues when close to triangles and when rendering triangles behind the camera. With the light in the middle of the scene, I couldn't compute a single face of the cube even after a minute. I also encountered issues with the light camera matrices, leading to strange transforms. As this happened towards the end of the project, I preferred to allocate my time to the report and optimizing the cone casting part over finishing fixing the renderer. This is a point to keep in mind as this has a great impact on the quality of the shadows.

V.2. Visual quality of the output

As the voxel grid changed, the fidelity of the output compared to the reference Blender render varied. For larger grid sizes, the shadows are more precise but also much darker. For smaller grids, the opposite is true, but we also see loads of incorrect artifacts on the wall. I would argue that for such a simple scene, a simpler direct light and ambient light-only model would result in much more accurate results while being faster, as seen in the direct light-only renders.

V.3. Impact of the Voxel Grid resolution

When comparing the output of the render with the reference Blender render, the results seem much darker on the floor and the shadows and much brighter on the roof. This issue is more visible as the resolution of the grid increases. I suspect this issue is hardly avoidable and is caused by the LOD when sampling. As the resolution increases, more and more voxels are sampled. However, only the voxels around the objects are filled in. This means that when sampling a face, only some of the voxels from the face's triangle count towards occlusion, and the resulting ambient occlusion is lowered and the ambient light diminished.

V.4. Render speed

As VCT can be implemented on top of a rasterizer, it can benefit from the high performance of the rendering for simple scenarios. This is clearly visible when only rendering direct light. Even for a 64x64x64 voxel grid, the render is twice as fast as the ray tracer, although with slightly pixelated shadows. Since the ray tracer only computed a single bounce, the times are not directly comparable with VCT, as the work is not equivalent. As the render for half cones is at worst 300 ms slower, it is likely that adding a bounce to the ray tracer would make it slower, but without testing, nothing is certain.

V.5. Memory Consumption

What Voxel Cone Tracing gains in speed, it loses in memory. The algorithm needs to store a shadow map as well as the whole scene in voxels. With a grid size of 512 voxels per axis, which appeared frequently in the other papers, the voxel grid would consume around 1.5 GB of memory. Looking at the size of modern 3D environments, it would be nearly impossible to store the entire voxel grid in memory.

VI. Conclusion

The scope set for this project was to discover and implement indirect illumination in a rasterizer using the Voxel Cone Tracing algorithm. This task was successfully completed except for the shadow map, which had to be worked around due to limitations of the renderer. While this algorithm is not comparable in workload to the ray tracer from Lab 2, as it did not use global illumination. Only using

the direct illumination of the algorithm led to much quicker render times, at the cost of high memory consumption. This high memory usage is the main downside of the algorithm, as it requires the whole scene to be voxelized to work correctly and a high voxel grid resolution to achieve more accurate results.

VII. References

- [1] 2026. Shadow mapping. Retrieved May 15, 2026 from https://en.wikipedia.org/wiki/Shadow_mapping
- [2] Alvar Trojenborg and Filip Northman. 2024. Voxel Cone Tracing Evaluation for Real-Time Applications. Retrieved from <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1886204&dswid=3088>
- [3] C. Crassin, F. Neyret, M. Sainz, S. Green, and E. Eisemann. 2011. Interactive Indirect Illumination Using Voxel Cone Tracing. In *Proc. Pacific Graphics*, 2011. Retrieved May 15, 2026 from https://research.nvidia.com/sites/default/files/pubs/2011-09_Interactive-Indirect-Illumination/GIVoxels-pg2011-authors.pdf
- [4] Cem Yuksel. 2022. *Interactive Graphics 16 - Shadow Mapping*. Youtube. Retrieved May 15, 2026 from <https://www.youtube.com/watch?v=Jhopq2lkzMQ>
- [5] FilipNorth. KEX-tracer. Retrieved May 15, 2026 from <https://github.com/FilipNorth/KEX-tracer/tree/Thesis-Result>
- [6] Spiky-73. DH2323 Project VCT. Retrieved May 15, 2026 from <https://github.com/Spiky-73/DH2323-Project-VCT/settings>